Technical architecture for air-gapped SDR development system

Introduction

Traditional radio systems were defined by their physical hardware, but modern Software Defined Radios (SDRs) shift that complexity into code. In defense and national security, this software handles everything from basic signal processing to sensitive encryption, making it the backbone of mission-critical communications and electronic warfare. Because this code is so vital, the environments where we write and test it are prime targets for state-sponsored cyberattacks and supply chain compromises.

To protect these assets, we use air-gapped workstations, systems physically disconnected from the internet and standard office networks. While an air-gap is the gold standard for security, it isn't invincible; modern threats can still bypass these barriers via infected USB drives, insider actions, or even electromagnetic leaks. This report outlines a hardened, multi-layered architecture designed to close the "Software Understanding Gap," ensuring our development process remains secure and auditable even against the most advanced persistent threats.

Security standards and compliance requirements

Building a secure development environment for national security isn't just about good engineering; it’s about following a proven blueprint. We rely on established federal and international standards to provide a mandatory baseline for every technical configuration. These frameworks ensure that our security measures are consistent, reliable, and most importantly - auditable, giving oversight teams a clear way to verify that the system is truly protected.

NIST SP 800-53 Rev. 5 framework

The National Institute of Standards and Technology (NIST) SP 800-53 serves as the essential rulebook for securing federal IT systems. For workstations handling military SDR code, we categorize the environment as "High Impact." This reflects just how vital the confidentiality and reliability of that source code are to national security, requiring us to implement a rigorous set of safety checks across the entire system.

To keep the environment locked down, we focus on four key areas. Access Control ensures that developers only have the specific permissions they need to do their jobs, while Audit and Accountability keeps a detailed, protected log of all system activity so we can trace every action if an issue arises. Configuration Management maintains a "known-good" state by preventing any unauthorized or accidental changes to the setup. Finally, System and Information Integrity tools

act as a constant watch, detecting and blocking threats like malware or unauthorized modifications before they can do damage.

DISA STIG and security requirements guides

While NIST 800-53 tells us *what* needs to be secured, DISA's Security Technical Implementation Guides (STIGs) provide the actual step-by-step instructions on *how* to do it. For an Ubuntu 24.04 workstation, the STIG translates high-level policies into hundreds of specific technical settings, covering everything from FIPS-validated encryption and Secure Boot to strict user permission limits. These guides turn abstract security goals into a concrete, repeatable configuration for the development team.

We prioritize these security requirements based on their risk level. Category I (Cat I) vulnerabilities are the most critical, as they could give an attacker immediate total control over the system. Category II and III issues represent medium to lower risks that could still be exploited or weaken our overall defense. To get the official "Authority to Operate" (ATO) on a military network, we must fix all Cat I and Cat II issues or prove that we have other strong protections in place to offset them.

CNSSI 1253 and national security systems

When a system handles National Security Information (NSI), it falls under CNSSI No. 1253. Think of it as a more mission-focused extension of NIST 800-53. Instead of assigning one overall impact rating to the entire system, CNSSI 1253 breaks risk down into three separate dimensions: confidentiality, integrity, and availability. This lets organizations define exactly how critical each aspect is (for example, C: High, I: High, A: High) and select security controls that directly match mission risk, rather than applying a one-size-fits-all model.

CNSSI 1253 also introduces specialized overlays for unique environments. In an air-gapped SDR development setup, that can mean stricter physical isolation, enhanced personnel vetting, and tightly controlled handling of removable media. These added layers recognize that protecting NSI isn't just about system configuration, it's about securing the entire operational ecosystem around it.

Architecture of the solution

The proposed workstation architecture is built to be its own fortress, creating a self-contained defense that doesn't rely on any external network security. We designed it with a "zero-trust" mindset, assuming that the underlying physical infrastructure, like an unmanaged switch, is fundamentally untrusted.

By treating every peripheral device as a potential entry point for an attack, the architecture shifts the focus to internal hardening. This approach ensures that even if the surrounding hardware is compromised, the core development environment remains isolated and secure.

A. Linux computer with secure boot, BIOS admin password, and encrypted LVM drive

The security of the operating system is rooted in the hardware and firmware layers. Ubuntu 24.04 LTS is implemented with a strict chain of trust that begins with the UEFI firmware.

Platform integrity and boot security

1. **UEFI Secure Boot**: Secure Boot is mandatory to prevent the loading of unapproved bootloaders or kernels. The system utilizes a Microsoft-signed shim binary that contains Canonical's public certificate, ensuring that only kernels signed by a trusted authority are executed

2. **BIOS Hardening**: Unauthorized modification of the boot sequence or security settings is prevented by a strong BIOS/UEFI administrative password. This setting is configured to require the password for both modification of settings and, optionally, for every boot event if the system remains in a sensitive area

3. **MOK Management**: If the developer requires custom kernel modules (common in SDR for specific FPGA or hardware interface drivers), these must be signed using a Machine Owner Key (MOK) and enrolled via the mokutil utility

   ```
   # Generate and enroll a custom key for kernel module signing
   openssl req -new -x509 -newkey rsa:2048 -keyout MOK.priv -outform
   DER -out MOK.der -nodes -days 36500
   sudo mokutil --import MOK.der
   ```

4. **Verification**: Auditors should verify the Secure Boot state using the following command:

   ```
   mokutil --sb-state
   # Expected Output: SecureBoot enabled
   ```

Data at rest protection with LUKS/LVM

To comply with NIST SC-28 (Protection of Information at Rest), the entire root filesystem and swap space are encrypted using Linux Unified Key Setup (LUKS)

- **Cryptographic Primitives**: The system should use AES-256 in XTS mode, which is the current industry standard for volume encryption

- **LVM Integration**: Encrypting at the LVM layer allows for flexible partition management while maintaining a single decryption event at boot

- **Verification**:

  ```
  # Check encryption status of partitions
  sudo cryptsetup status dm-0
  ```

```
# Expected Result: type: LUKS2, cipher: aes-xts-plain64
```

B. Developer user with limited privileges

The architecture strictly separates administrative duties from development activities to prevent inadvertent configuration changes and mitigate the impact of account compromise

User account configuration

- **Standard Account**: The developer operates using a standard user account with no permanent sudo privileges
- **Separation of Duties**: An administrative account is created during installation but is only used for system maintenance
- **Restrictive Settings**: The standard user is prevented from installing new software or modifying global system parameters. This is audited by checking the members of the sudo and admin groups

```
# Verify group membership
grep -E 'sudo|admin' /etc/group
# Standard users should NOT be listed here
```

C. WiFi and Bluetooth disabled

Air-gap integrity is enforced by the absolute removal of wireless communication capabilities. This prevents the system from being discovered by nearby devices or used as a pivot for unauthorized data transfer

Radio suppression methodology

1. **Kernel Module Blacklisting**: Drivers for common wireless and Bluetooth chipsets are prevented from loading at boot

```
# Create /etc/modprobe.d/blacklist-radio.conf
blacklist iwlwifi
blacklist bluetooth
blacklist btusb
blacklist btrtl
blacklist uvcvideo # Optional: disable camera if present
```

2. **Hardware RF Block**: Use the rfkill utility to enforce a software-level block that prevents the radio hardware from receiving power

```
sudo rfkill block all
# Verify state
rfkill list
# Expected: Soft blocked: yes, Hard blocked: yes (if physical
```

```
switch exists)
```

3. **Service Masking**: Systemd services for network management and Bluetooth are masked to prevent them from being started by other processes

```
sudo systemctl stop bluetooth.service
sudo systemctl mask bluetooth.service
```

D. Only authorized devices are able to be connected to USB ports

USB ports represent the most significant physical attack surface for an air-gapped machine. The architecture implements USBGuard to verify the identity of every connected device before allowing it to interact with the kernel

USB authorization with USBGuard

USBGuard uses a rule-based engine to match device attributes such as Vendor ID, Product ID, and unique serial numbers against a whitelist

1. **Rule Generation**: A restrictive policy is generated based on the minimum required peripherals (e.g., a specific keyboard and mouse)

```
# Generate initial policy and enforce whitelisting
sudo usbguard generate-policy > /etc/usbguard/rules.conf
# Set default target to block in /etc/usbguard/usbguard-
daemon.conf
ImplicitPolicyTarget=block
```

2. **Attribute Matching**: To prevent "BadUSB" attacks where a device spoofs its class (e.g., a keyboard acting as a network adapter), rules should include interface class validation

```
# Example allow rule for a specific authorized peripheral
allow id 046d:c52b name "Logitech Receiver" with-interface
03:01:01
```

3. **Hardware Persistence**: USBGuard tracks the physical port "via-port" to ensure that authorized devices are only used in their designated locations, further complicating bypass attempts

E. LAN communication restricted with no MAC cloning

In a network utilizing an unmanaged Layer 2 switch, standard security features like DHCP Snooping or Port Security are unavailable. The workstation must therefore implement its own protection against ARP poisoning and MAC spoofing

Host-side network hardening

1. **Static ARP Mapping**: The workstation ignores ARP resolution entirely for critical peers (e.g., a local waveform repository or another development terminal) and utilizes manually

configured static entries

```
# Add a permanent ARP entry
sudo ip neighbor add 192.168.1.10 lladdr 00:e0:1e:b4:12:42 nud
permanent dev eth0
# Verify mapping
arp -va
# Expected Result: (192.168.1.10) at 00:e0:1e:b4:12:42 [ether]
PERM on eth0
```

2. **Kernel ARP Controls**: Configure the kernel to ignore ARP requests for IP addresses not explicitly configured on the incoming interface, preventing certain types of network discovery probes

```
# In /etc/sysctl.d/99-arp-hardening.conf
net.ipv4.conf.all.arp_ignore = 1
net.ipv4.conf.all.arp_announce = 2
```

3. **Link Layer Filtering**: The architecture utilize ebtables-nft or raw nftables to drop all Ethernet frames that do not originate from the allowed source MAC addresses on the local segment

F. Authorized USB pendrives must be encrypted

When information must be transferred across the air-gap (the "Sneakernet" scenario), the portability of data on USB media creates a risk of disclosure if the physical device is lost or stolen

Removable media standards

- **Mandatory Encryption**: Organizationally issued transfer media must be encrypted using FIPS-validated cryptography
- **LUKS for Removable Media**: USB drives are formatted with a LUKS container. The developer must authenticate to the drive before the encrypted volume is mapped and the filesystem mounted

```
# One-time setup for authorized media
sudo cryptsetup luksFormat /dev/sdb1
sudo cryptsetup open /dev/sdb1 transfer_vol
sudo mkfs.ext4 /dev/mapper/transfer_vol
```

- **Mount Point Monitoring**: The operating system is configured to log all mount events and file access on removable media via auditd

G. Total internet isolation on unmanaged infrastructure

Even if a malicious router or a misconfigured laptop attempts to act as a gateway on the local switch, the workstation must be structurally incapable of routing traffic to any destination outside the trusted local subnet

Logical air-gap with nftables

The system implements a local-only firewall policy using nftables, the modern replacement for iptables in the Linux kernel

1. **Subnet Isolation**: Define a specific table and chain that only permits traffic within the 192.168.1.0/24 range and explicitly drops everything else

   ```
   table inet isolation {
       chain input {
           type filter hook input priority 0; policy drop;
           ip saddr 192.168.1.0/24 accept
           iifname "lo" accept
       }
       chain output {
           type filter hook output priority 0; policy drop;
           ip daddr 192.168.1.10 accept # Explicit peer
           ip daddr 192.168.1.0/24 accept
           oifname "lo" accept
       }
   }
   ```

2. **Null-Routing and Gateway Removal**: The default gateway is removed from the routing table, and the configuration is set to manual (no DHCP) to prevent a rogue router from assigning a new gateway

   ```
   # Remove any existing default routes
   sudo ip route del default
   # Verify routing table is local-only
   ip route show
   ```

3. **DNS Suppression**: Resolve configuration is set to a non-functional address or points only to a trusted local DNS server if required for internal resource naming.

H. Local data vault for project artefacts

To provide defense-in-depth for SE and Waveform intellectual property, developers utilize a secondary layer of encryption for their active project directories

gocryptfs per-user vaults

gocryptfs provides file-based encryption that runs in user-space via FUSE, allowing users to secure their sensitive code without requiring administrative privileges for day-to-day operations

- **Encryption at Rest**: Files are stored in an encrypted "cipher" directory. When mounted with the correct password, a "plain" directory appears where the developer can work

- **Security Advantages**: Unlike whole-disk encryption, the vault can be unmounted when not in use, protecting the data even if the logged-in session is compromised

```
# Mount the vault for development
gocryptfs ~/sdr_project_enc ~/sdr_project_workspace
# Unmount when work session ends
fusermount -u ~/sdr_project_workspace
```

I. Access control and configuration lockdown

User actions are further restricted through Discretionary Access Control (DAC) and Polkit rules to ensure that standard developers cannot tamper with system-wide security settings

System-wide restrictions

1. **Configuration File Integrity**: All files in /etc are owned by root with 0644 or more restrictive permissions. Write access is strictly prohibited for standard users

2. **Polkit Lockdown**: Modern Linux desktops use Polkit to manage privileged actions. A custom rule is implemented to require administrative authentication for any network or power management changes

   ```
   // /etc/polkit-1/rules.d/00-developer-lockdown.rules
   polkit.addRule(function(action, subject) {
       if (action.id.indexOf("org.freedesktop.NetworkManager.") == 0
   ||
           action.id.indexOf("org.freedesktop.login1.reboot") == 0) {
           return polkit.Result.AUTH_ADMIN;
       }
   });
   ```

3. **Home Directory Isolation**: Users are prevented from browsing other developers' files through the enforcement of 0700 permissions on home directories

J. Admin event logging with auditd

The Linux Audit System (auditd) is the primary tool for recording security-relevant events. It operates by hooking into kernel-level system calls, providing a tamper-resistant record of all system activity

Comprehensive audit configuration

1. **Syscall Monitoring**: Audit rules are applied to track the execution of binaries (execve), modifications to file permissions (chmod, chown), and attempts to access sensitive files like /etc/shadow

2. **USBGuard Integration**: The USBGuard daemon is configured to offload its authorization events to the LinuxAudit backend, centralizing all hardware-related security events in the main audit log

3. **Immutability Mode**: To prevent a compromised root account from hiding its tracks, the audit system is set to immutable mode. This ensures that the rules cannot be changed

without a full system reboot, which is a highly visible event

```
# Final rule in /etc/audit/rules.d/audit.rules
-e 2
```

4. **Reporting**: Auditors use aureport and ausearch to review events periodically

```
# Generate a summary of system login attempts
sudo aureport -au
# Search for all USB device connection events
sudo ausearch -k usb_device
```

Compliance explanation

The architecture above is designed to satisfy the rigorous requirements of a High Impact system categorization under NIST SP 800-53 Rev. 5 and the associated DISA STIG benchmarks

Mapping architecture to federal standards

The following table provides a direct mapping between the proposed architectural components and the specific controls and CCIs they satisfy

Architecture Topic	NIST 800-53 Control	DISA STIG / CCI Number	Implementation Mechanism
Boot Integrity	CM-3, SI-7	CCI-002165, V-270651	UEFI Secure Boot, BIOS Password, AIDE.
Data Privacy	SC-28, MP-6	UBTU-24-90890, V-230224	LUKS full-disk encryption, encrypted swap.
Access Control	AC-2, AC-3, AC-6	CCI-002165, V-270748	Restricted sudoers, Polkit network lockdown.
Wireless Control	AC-18	V-270656	Kernel blacklisting, rfkill soft block.
USB Security	IA-3, MP-7	CCI-001958, V-244548	USBGuard whitelisting by ID and serial.
LAN Hardening	SC-7, SC-8	CCI-002019, V-270656	Static ARP, local-only nftables rules.
Media Encryption	MP-4, MP-5	CCI-000162, V-202028	Mandatory LUKS on authorized pendrives.
Logging	AU-2, AU-3, AU-9	CCI-000169, V-230402	auditd with immutable rules, USBGuard logs.

Table: Compliance mapping of SDR development workstation security controls

Detailed control discussion for specific clauses:

- **AC-18 (Wireless Access)**: The requirement is to authorize wireless access to the system prior to allowing connections. This architecture exceeds the requirement by implementing

a "Deny All" policy enforced at the kernel module level, which physically prevents the instantiation of the wireless interface

- **SC-7 (Boundary Protection)**: Boundary protection usually refers to firewalls at the network perimeter. In this air-gap architecture, the "boundary" is shifted to the host's NIC. By utilizing nftables to drop all traffic not explicitly local, the workstation creates an internal enclave that is resilient to Layer 2 infrastructure compromise

- **AU-2 (Audit Record Generation)**: The system must produce records with sufficient information to establish what type of event occurred, when it happened, and who initiated it. The use of auditd in enriched mode resolves UIDs to human-readable names and logs the outcome (success/fail) of all privileged system calls, satisfying both technical and administrative audit needs

Pentesting and validation scenarios

To ensure the technical efficacy of the implemented controls, the system must undergo periodic validation using penetration testing methodologies. These tests simulate adversarial behaviors and measure the system's defensive response

Scenario 1: Malicious router and gateway impersonation

- **Attack Vector**: An attacker connects a laptop to the unmanaged switch, enables IP forwarding, and uses macchanger to clone the MAC address of a trusted peer

- **Test Procedure**: The attacker laptop broadcasts ARP replies claiming to be the gateway for the 192.168.1.0/24 subnet

- **Verification Command**:

```
# On the development workstation
arp -va
```

- **Pass Criteria**: The ARP table must remain static. The workstation must ignore the attacker's ARP replies because the original peer's MAC is mapped with the PERM flag

- **Expected Result**: Communication with the genuine peer continues; packets to the attacker's "gateway" are dropped

Scenario 2: USBGuard bypass with device class spoofing

- **Attack Vector**: An attacker uses a specialized microcontroller (e.g., Teensy or Rubber Ducky) to present itself as a standard keyboard (allowed by policy) but attempts to register a hidden mass storage or network interface

- **Test Procedure**: Insert the device and monitor the system response

- **Verification Commands:**

```
sudo ausearch -k usb_device -ts recent
```

```
usbguard list-devices | grep block
```

- **Pass Criteria**: USBGuard must detect that the device provides multiple interfaces. Since the policy only allows a specific HID interface hash, the device must be blocked or rejected
- **Expected Result**: No additional storage or network device appears in lsblk or ip addr

Scenario 3: Unauthorized network exfiltration attempt

- **Attack Vector**: A malicious script on the workstation attempts to "phone home" to a hardcoded external IP address, simulating a successful initial foothold
- **Test Procedure**: Attempt to ping or establish a socket to an IP outside the local subnet
- **Verification Command**:

```
ping -c 4 8.8.8.8
curl --connect-timeout 5 http://203.0.113.5
```

- **Pass Criteria**: All outbound packets to non-local addresses must be dropped by the nftables output chain
- **Expected Result**: "Network is unreachable" or "Operation not permitted."

Scenario 4: Privilege escalation via sudo configuration

- **Attack Vector**: A standard developer account attempts to modify a system-wide configuration file (e.g., /etc/passwd) using a potential oversight in sudo permissions
- **Test Procedure**: Attempt to run an unauthorized command with sudo
- **Verification Command**:

```
sudo vi /etc/ssh/sshd_config
```

- **Pass Criteria**: The system must deny the request. The attempt must be logged by auditd with the sudo_usage key
- **Expected Result**: User is prompted for a password they do not have, or receives "user is not in the sudoers file." Audit log shows a USER_AUTH fail

Scenario 5: Unauthorized wireless radio activation

- **Attack Vector**: A user attempts to re-enable the WiFi radio using the nmcli or rfkill utilities to establish a hidden data bridge
- **Test Procedure**: Attempt to unblock and start the wireless interface
- **Verification Command**: ```bash sudo rfkill unblock wifi nmcli radio wifi on

- **Pass Criteria**: Since the kernel modules are blacklisted, the interface cannot be initialized. Furthermore, standard users are blocked from this action by Polkit
- **Expected Result**: "Error: NetworkManager is not running" (if service is masked) or "Access denied."

Conclusion

The security architecture in this report offers a complete, defense-in-depth solution for military-grade SDR development. By combining hardware-level protections with strict user controls and unchangeable auditing, the system creates a level of structural isolation that goes far beyond a simple physical air-gap. This approach ensures that the development environment is not just disconnected, but actively hardened against sophisticated intrusion attempts.

This setup is fundamentally secure for four key reasons.

First, hardware-anchored trust through UEFI Secure Boot and BIOS passwords prevents the system from being compromised during startup.

Second, host-centric network isolation using static ARP and nftables turns each workstation into a logical island, neutralizing common network attacks.

Third, strict USBGuard whitelisting by serial number blocks the most common air-gap threat: infected removable media.

Finally, immutable logging ensures that any attempt to bypass these defenses is permanently recorded for forensic analysis, providing a robust, auditable framework that protects the critical intellectual property essential to national defense.